YAZILIM GELİŞTİRME YAŞAM DÖNGÜSÜ (SDLC)

1. SDLC NEDİR? (Neden Bir "Yaşam Döngüsü"?)

SDLC (Software Development Life Cycle), bir yazılımın **fikir aşamasından başlayıp, kullanımdan kaldırılana kadar** geçen süreci yöneten, adım adım ilerleyen **çerçeve (framework)** ve **metodolojiler bütünüdür**.

> SDLC, bir ev inşa etme sürecidir. Proje başlamadan mimari çizim (analiz), zemin kazma (tasarım), duvar örme (geliştirme), sıva-boya (test) ve son olarak anahtar teslim (dağıtım) aşamaları vardır. Bu aşamalardan birini atlarsanız ev çöker.

SDLC'nin amacı; **kaliteli, bütçeye uygun, zamanında teslim edilen** ve **kullanıcı ihtiyacını karşılayan** bir yazılım üretmektir.

---

2. SDLC'NİN 6 TEMEL AŞAMASI

Hangi modeli seçersen seç, aşağıdaki aşamalar **mutlaka** yaşanır. Sadece bu aşamaların **sıralaması ve tekrarlanma sıklığı** değişir.

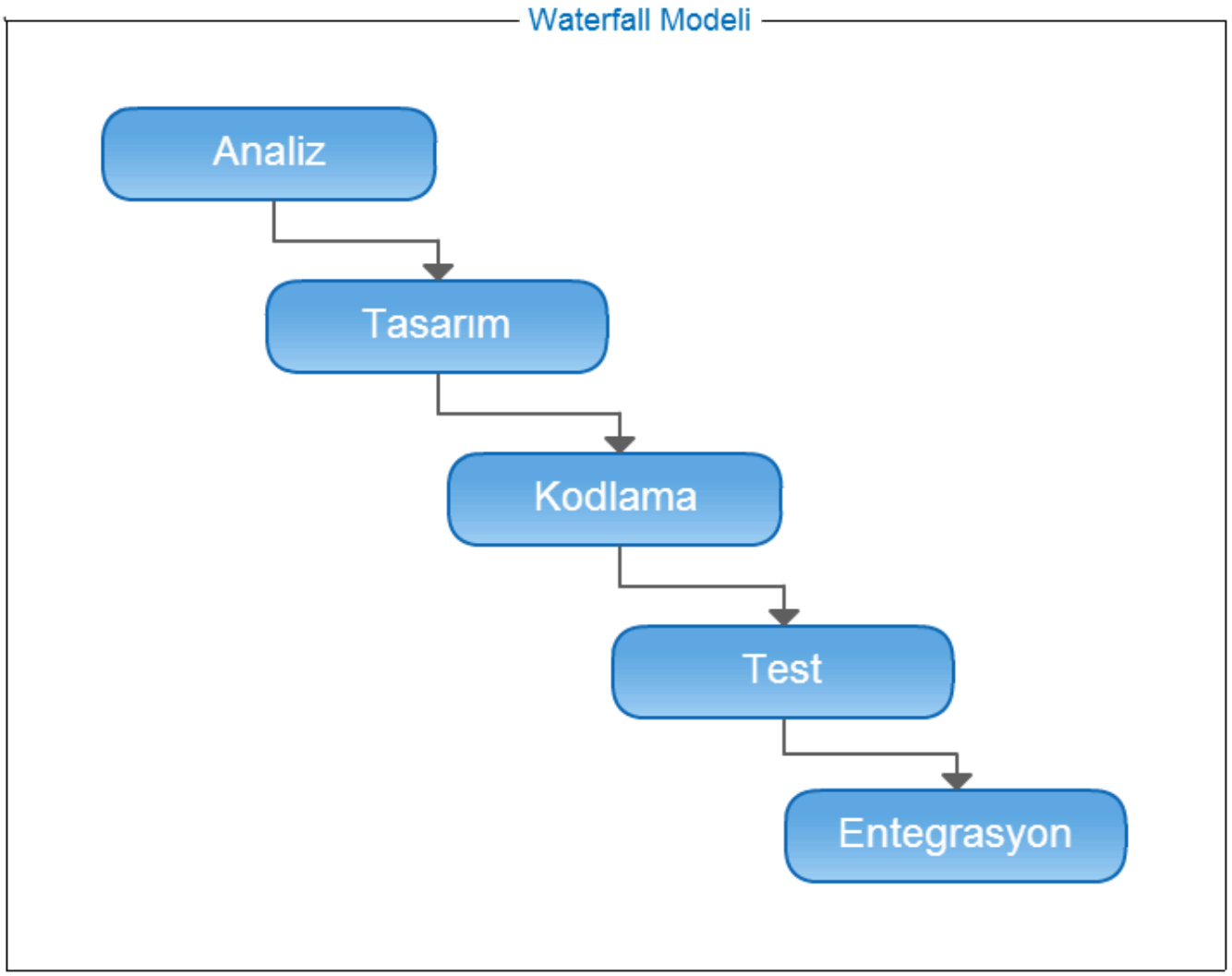
Aşama	Ne Yapılır?	Çıktısı (Teslim Edilen) Nedir?
1. Gereksinim Analizi (Planlama)	Müşteri/iş birimi ile görüşülür. "Ne istiyoruz?" sorusuna cevap aranır. Sistem Analisti burada devreye girer.	Gereksinim Dokümanı (SRS), Proje Planı
2. Sistem Tasarımı (Design)	Yazılım mimarisi çizilir. Hangi veritabanı, hangi dil, hangi altyapı? Ekran taslakları (mockup) yapılır.	Mimari Tasarım Dokümanı, Veritabanı Şeması, API Tasarımı
3. Geliştirme (Implementation)	Kodlamanın yapıldığı aşamadır. Geliştiriciler ekranları ve arka plan kodlarını yazar.	Çalışan kaynak kodları (Source Code)
4. Test (Testing)	Yazılımın hataları (bug) aranır. Birim test, entegrasyon test, sistem testi yapılır.	Hata Raporları, Test Senaryoları
5. Dağıtım (Deployment)	Yazılım canlı ortama (sunucuya/markete) alınır. Kullanıcılar erişmeye başlar.	Canlıda çalışan sistem, Kullanıcı Eğitim Dokümanları
6. Bakım ve Destek (Maintenance)	Canlıdaki hatalar düzeltilir, yeni özellik eklenir, performans iyileştirilir.	Güncelleme notları, Versiyon yamaları

---

3. SDLC MODELLERİ: SUYU NASIL TAŞIRSIN? (Waterfall, Agile, Scrum)

İşte can alıcı nokta! Aynı aşamaları, farklı **felsefelerle** uygulayabilirsin. Bu felsefelere **"Yazılım Geliştirme Metodolojisi"** denir.

A) WATERFALL (Şelale Modeli) – Klasik ve Sıralı



Mantığı: Suyun yukarıdan aşağıya akması gibi, her aşama **tamamlanmadan** bir sonraki aşamaya **geçilmez**. Geri dönüş yoktur (ya da çok zordur).

- Analiz (100% tamam) > Tasarım (100%) > Geliştirme > Test > Dağıtım

Artıları (İyi Yönleri):

- **Basit ve anlaşılır:** Herkes ne zaman ne yapacağını bilir.
- **Disiplinli:** Aşamalar nettir, kaytarma olmaz.
- **Dokümantasyon zengin:** Her aşama ayrıntılı belgelenir, bu da ileride sistem değişikliğinde başvuru kaynağı olur.
- **Büyük ve kritik projelerde güvenlidir** (Örn: Uzay araçları, sağlık cihazları – hata telafisi yoktur).

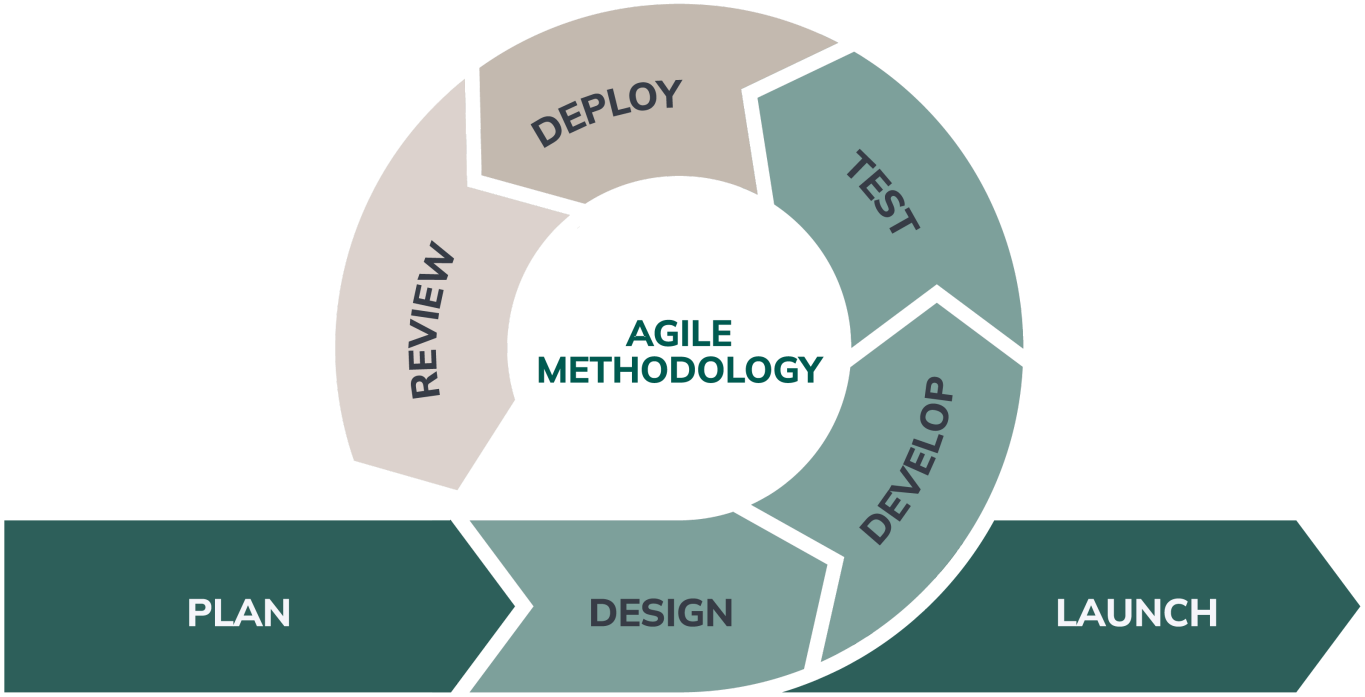
Eksileri (Kötü Yönleri):

- **Esneklik sıfırdır:** Projenin ortasında müşteri "Ben bu ekranı beğenmedim, değiştirelim" derse, başa dönmek aylar alır.
- **Geç teslim:** Müşteri yazılımı **en son aşamada** görür. O zamana kadar beklentiler değişmiş olabilir.

- **Risk yönetimi zayıftır:** Büyük hatalar en son test aşamasında ortaya çıkar, düzeltmesi çok pahalıdır.
- **Günümüz hızına uygun değildir** (Pazarda 1 hafta gecikme ölümdür).

> **Pastane Örneğinde Waterfall kullansaydık:** Ayşe Hanım 6 ay boyunca hiçbir şey görmeden bekleyecek, sonunda gelen yazılımda "Ben pastaların fotoğrafını büyük görmek istemiştim, sen neden küçük koydun?" diyerek kriz çıkaracaktı.

B) AGILE (Çevik) – Günümüzün Kurtarıcısı



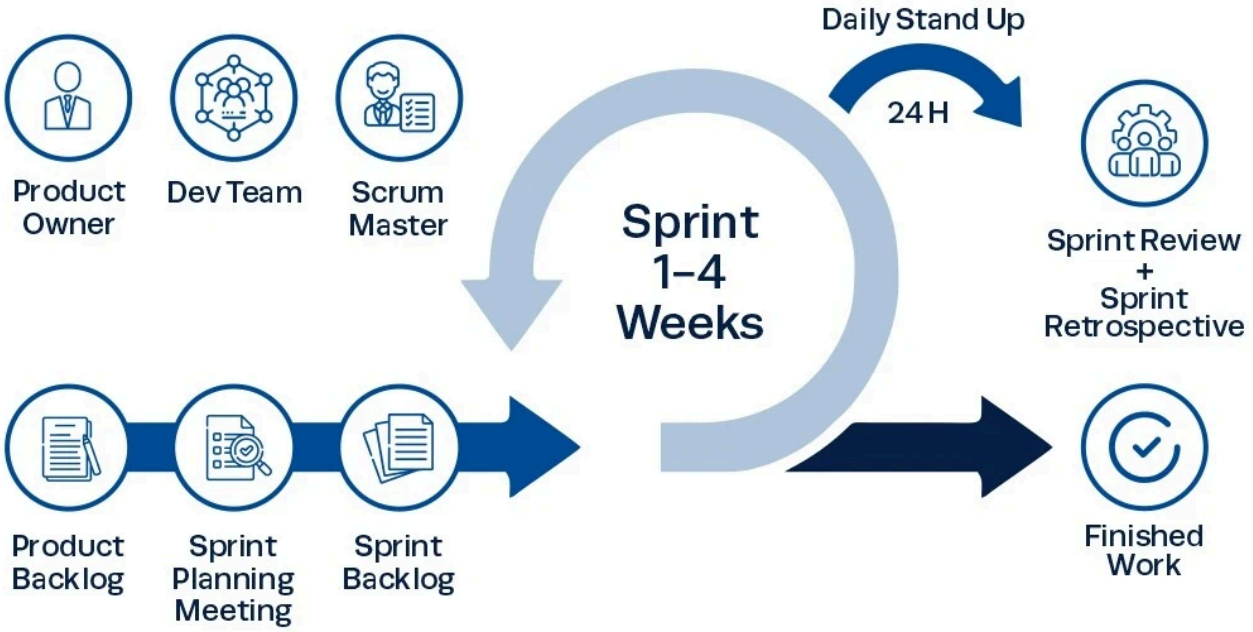
Mantığı: Waterfall'a tepki olarak doğmuştur. İnsanların ve ihtiyaçların hızla değiştiğini kabul eder. "Büyük resmi değil, küçük küçük teslim et, geri bildirim al, düzelt" der.

Agile, aslında **bir manifesto (beyanname)**'dir. 4 temel değeri vardır:

1. İnsan ve etkileşim = Süreç ve araçlardan önemlidir.
2. Çalışan yazılım = Kapsamlı dokümandan önemlidir.
3. Müşteri ile iş birliği = Sözleşme pazarlığından önemlidir.
4. Değişime karşılık vermek = Plana sadık kalmaktan önemlidir.

Agile **bir metodoloji değil, bir şapkadır**. Bu şapkanın altında en meşhur takım ise **Scrum**'dur.

C) SCRUM – Agile'ın En Popüler Takım Oyunu



Mantığı: Bu bir **çerçevedir**. İşi, **1-4 haftalık "Sprint"** adı verilen kısa zaman dilimlerine böler. Her Sprint sonunda **çalışan, test edilmiş, kullanılabilir** bir ürün parçası teslim edilir.

Scrum'un 3 Temel Unsuru:

Rol	Açıklama
Ürün Sahibi (Product Owner)	İşin patronudur. Müşteriyi temsil eder, öncelikleri belirler. "Önce sipariş ekranı, sonra raporlar" der.
Scrum Master	Takımın koçudur. Sürecin doğru işlemesini sağlar, engelleri kaldırır. (Patron değil, hizmetkârdır.)
Geliştirme Takımı	Kodlayıcılar, testçiler, analistler. Her Sprint için seçilen işleri bitirmekle yükümlüdür.

Scrum Akışı (Sprint Döngüsü):

1. **Backlog (Yapılacaklar Listesi):** Tüm özellikler buraya yazılır.
2. **Sprint Planlama:** Takım, bu listeden en önemlilerini seçer (örnek: 2 haftada bitecek kadar).
3. **Günlük Scrum (Daily Stand-up):** Her gün 15 dakika toplanıp "Dün ne yaptım, bugün ne yapacağım, engel var mı?" denir.
4. **Sprint Review (Gözden Geçirme):** Sprint sonunda biten özellikler müşteriye gösterilir, geri bildirim alınır.
5. **Sprint Retrospective (Retro):** Takım kendi içinde "Gelecek sefer nasıl daha iyi yaparız?" diye konuşur.

Agile/Scrum Artıları:

- **Değişime açık:** Müşteri 2 haftada bir geri bildirim verir, yanlışlar hemen düzeltilir.
- **Hızlı geri dönüş:** Yazılımın bir kısmı erken devreye alınabilir (Ayşe Hanım 1. Sprint'te sipariş panelini kullanmaya başlar).
- **Motivasyon yüksek:** Takım kendi işini planlar, baskı yerine sahiplenme olur.
- **Risk azalır:** Hata en geç 1 ay içinde yakalanır, maliyeti düşüktür.

Agile/Scrum Eksileri:

- **Disiplin şarttır:** Her gün toplantı, her 2 haftada teslim çok yorucu olabilir.
- **Dokümantasyon zayıftır:** "Çalışan kod" odaklıdır, ileride sistemi devralan yeni bir takım ne yapacağını anlamakta zorlanabilir.
- **Her projeye uymaz:** Örneğin, uçak pilotu yazılımı veya banka ana çekirdek sistemi gibi kritik projelerde "deneme yanılma" kabul edilemez. Burada Waterfall daha güvenlidir.
- **Müşteri bağımlılığı:** Müşteri kararsızsa, takım her Sprint'te farklı yöne savrulur, proje biter.

---

4. PEKİ HANGİSİNİ SEÇMELİYİM? (Analist Gözüyle Karşılaştırma)

Kriter	Waterfall	Agile / Scrum
Proje Büyüklüğü	Büyük, karmaşık, süresi 1 yıldan uzun	Orta/küçük, dinamik
Gereksinim Netliği	%100 net ve değişmeyecek	Net değil, sürekli evrilecek
Müşteri Katılımı	Sadece başta ve sonda	Her Sprint'te sürekli
Dokümantasyon	Çok detaylı	Minimum, kod ağırlıklı
Risk	Düşük teknoloji riski, yüksek iş riski	Düşük iş riski, yüksek yönetim disiplini riski

---

ÖZET

- **SDLC** = Yazılımın ana haritası (6 aşama).
- **Waterfall** = Sırayla, sağlam, ama katı. (Geçmişin güvencesi)
- **Agile** = Esnek, hızlı, insan odaklı. (Bugünün kurtarıcısı)
- **Scrum** = Agile'ı uygulamak için en yaygın taktik. (Sprint'lerle ilerle)

> **Nasıl seçebilirim?;** Hangi modeli seçersen seç, **amaç müşterinin sorununu çözmek**. Eğer müşteri "Ne istediğimi çok iyi biliyorum, asla değişmez" diyorsa Waterfall, "Fikrim var ama göre göre karar veririm" diyorsa Agile/Scrum kullan.

• Bir e-ticaret sistemi bileşenlerine ayır. "Bir sistemi parçala ve yeniden tanımla" • Aynı projeyi (ör: yemek sipariş uygulaması) o waterfall vs agile olarak planlama • "Hangisi neden daha iyi?"

E-Ticaret Sisteminin Bileşenlerine Ayrılması, Parçalanması ve Yeniden Tanımlanması

1. BİLEŞENLERİNE AYIRMA (Decomposition)

Bir e-ticaret sistemini üst düzey bileşenlerine ayıralım:

1.1 Ana Bileşenler

#	Bileşen	Görev
1	**Kullanıcı Yönetimi**	Kayıt, giriş, profil, yetkilendirme
2	**Katalog Yönetimi**	Ürün, kategori, stok, fiyat
3	**Arama & Keşif**	Arama, filtreleme, öneri
4	**Sepet & Sipariş**	Sepet, sipariş oluşturma, durum takibi
5	**Ödeme**	Kart, havale, kapıda ödeme, 3D Secure
6	**Kargo & Lojistik**	Kargo seçimi, takip, iade
7	**Envanter**	Stok rezervasyonu, tedarik
8	**Bildirim**	E-posta, SMS, push
9	**Admin Panel**	Ürün, sipariş, kullanıcı yönetimi
10	**Analitik**	Raporlama, dashboard, KPI

---

2. PARÇALAMA (Breakdown)

Her bileşeni alt parçalara bölelim:

2.1 Kullanıcı Yönetimi

- Kayıt (e-posta, sosyal, misafir)
- Giriş / Oturum (JWT, OAuth)
- Profil (adres, ödeme yöntemi)
- Rol & yetki (müşteri, satıcı, admin)

2.2 Katalog

- Ürün CRUD
- Kategori ağacı
- Varyant (renk, beden)
- Medya (görsel, video)
- Fiyat & kampanya

2.3 Sepet & Sipariş

- Sepet ekle/çıkar/güncelle
- Kupon uygulama
- Sipariş oluşturma
- Sipariş durumu (beklemede → hazırlanıyor → kargoda → teslim)

2.4 Ödeme

- Ödeme sağlayıcı entegrasyonu (Stripe, iyzico)
- 3D Secure
- İade / iptal
- Fatura

2.5 Kargo

- Kargo firması entegrasyonu
- Takip numarası
- İade süreci

2.6 Envanter

- Stok giriş/çıkış
- Rezervasyon
- Düşük stok uyarısı

2.7 Bildirim

- Şablon yönetimi
- Tetikleyiciler (sipariş, kargo, kampanya)

2.8 Analitik

- Satış raporu
- Kullanıcı davranışı
- Dönüşüm oranı

---

3. YENİDEN TANIMLAMA (Redefinition)

Parçaları **domain-driven** ve **modüler** bakışla yeniden tanımlayalım:

3.1 Yeniden Tanımlanmış Bileşenler (Domain Bazlı)

Domain	Sorumluluk	Sınır (Bounded Context)
Identity	Kimlik, oturum, yetki	Kullanıcı verisi
Catalog	Ürün bilgisi	Ürün, kategori
Pricing	Fiyat, kampanya	Fiyat kuralları
Cart	Geçici sepet	Oturum bazlı

Domain	Sorumluluk	Sınır (Bounded Context)
Order	Sipariş yaşam döngüsü	Sipariş kaydı
Payment	Ödeme işlemi	Finansal işlem
Inventory	Stok	Depo verisi
Shipping	Lojistik	Kargo verisi
Notification	İletişim	Mesaj şablonları
Analytics	Raporlama	Olay verisi

3.2 Yeni Tanım Örneği

> **Order Domain:** "Müşterinin satın alma niyetinin, ödeme ve stok doğrulaması sonrası kalıcı bir sözleşmeye dönüştüğü; iptal, iade ve takip süreçlerini yönettiği bağlam."

Bu tanım, eski "Sepet & Sipariş" bileşenini **Cart** ve **Order** olarak ayırır; çünkü sepet geçici, sipariş kalıcıdır.

---

4. WATERFALL PLANI

4.1 Varsayımlar

- Kapsam baştan net
- Değişiklik az
- 12 ay süre, 6 kişilik ekip

4.2 Fazlar

Faz	Süre	Çıktı
1. Gereksinim	2 ay	SRS dokümanı, use-case
2. Tasarım	2 ay	Mimari, DB şeması, UI mockup
3. Geliştirme	4 ay	Modüller sırayla kodlanır
4. Test	2 ay	Birim, entegrasyon, UAT
5. Dağıtım	1 ay	Canlıya alma
6. Bakım	1 ay	Hata düzeltme

4.3 Sıralı Geliştirme

```
Gereksinim → Tasarım → Kod → Test → Dağıtım → Bakım
   ↓            ↓        ↓      ↓        ↓         ↓
 (donmuş)   (donmuş)  (donmuş)(donmuş)(donmuş)  (donmuş)
```

4.4 Riskler

- Değişiklik maliyeti yüksek
- Geç geri bildirim
- Entegrasyon sorunları geç fark edilir

---

5. AGILE PLANI

5.1 Varsayımlar

- Kapsam esnek
- 2 haftalık sprintler
- 6-8 kişilik cross-functional ekip
- MVP öncelikli

5.2 Ürün Backlog (Öncelik Sırası)

#	User Story	Puan	Sprint
1	Kullanıcı kayıt/giriş	5	1
2	Ürün listeleme	8	1-2
3	Ürün detay	5	2
4	Sepete ekleme	5	3
5	Sipariş oluşturma	8	3-4
6	Ödeme entegrasyonu	13	4-5
7	Sipariş takibi	5	5
8	Admin panel (ürün)	8	6
9	Kargo entegrasyonu	8	6-7
10	Bildirim	5	7
11	Analitik dashboard	8	8
12	İade süreci	8	8-9

5.3 Sprint Yapısı

```
Sprint 0: Setup, CI/CD, mimari iskelet
Sprint 1–2: MVP (kayıt, ürün, sepet)
Sprint 3–4: Sipariş + ödeme
Sprint 5–6: Admin + kargo
Sprint 7–8: Bildirim + analitik
Sprint 9: İade + polish
```

5.4 Meetingler

- **Daily Standup:** 15 dk
- **Sprint Planning:** 2 saat
- **Sprint Review:** 1 saat
- **Retrospective:** 1 saat

5.5 Agile Prensipler

- Çalışan yazılım > dokümantasyon
- Müşteri iş birliği > sözleşme
- Değişime yanıt > plana bağlılık

---

6. KARŞILAŞTIRMA

Kriter	Waterfall	Agile
Kapsam	Sabit	Esnek
Teslimat	Sonda	Her sprint
Geri bildirim	Geç	Erken
Risk	Yüksek	Düşük
Dokümantasyon	Ağır	Hafif
Uygun proje	Net gereksinim	Belirsiz gereksinim
E-ticaret için	❌ Uygun değil	✅ Uygun

---

7. ÖNERİ

E-ticaret sistemleri **dinamik pazar**, **sürekli değişen gereksinim** ve **hızlı geri bildirim** gerektirdiği için:

> **Hibrit yaklaşım önerilir:**
>
> - **Waterfall:** Altyapı, güvenlik, ödeme entegrasyonu (kritik, net)
> - **Agile:** UI/UX, kampanya, öneri motoru, analitik (esnek, deneysel)

Böylece hem **stabilite** hem **çeviklik** sağlanır.